Solution File Format

1. Naming the files

Solutions are plain-text files with the `.dat` extension. For the instance:

```
MPVRP_001_s48_d1_p1.dat
```

the preferred solution name is:

```
Sol_MPVRP_001_s48_d1_p1.dat
```

The shorter name `Sol_001.dat` is also accepted. A submission archive may organize files in folders, but it must include one solution for every instance from `001` to `150`.

2. Describing a vehicle schedule

Every used vehicle is represented by two matching lines. Leave an empty line before the next vehicle.

Route line

```
ID: Garage - Depot [Load] - Station (Delivery) - ... - Garage
```

This line follows the vehicle from departure to return:

- a **garage** is written with its identifier;
- a **depot** is followed by the quantity loaded in square brackets;
- a **station** is followed by the quantity delivered in parentheses.

Identifiers are local to their category. Depot 1, garage 1, and station 1 are therefore three different locations.

Product and cost line

```
ID: Product(CumulativeCost) - Product(CumulativeCost) - ...
```

This second line gives the vehicle's product configuration and cumulative transition cost at every step of the route. Product identifiers start at `0` in solution files and range from `0` to `NbProducts - 1`.

The first value is the vehicle's initial product configuration. At every depot, the next value is the product being loaded. This is also where any preparation and loading-related transition cost is added. The amount comes from the directed matrix using the previous configuration and the newly loaded product. It therefore applies before the first delivery trip as well as between later trips. In the current benchmark, loading the same product again adds no transition cost because the matrix diagonal is zero.

The route line and the product line must have exactly the same number of elements: every visited location has one corresponding product and cumulative cost.

3. Example

```
1: 1 - 1 [1344] - 2 (1344) - 1
1: 0(0.0) - 0(0.0) - 0(0.0) - 0(0.0)

2: 1 - 1 [8947] - 1 (4278) - 2 (2350) - 3 (2319) - 1
2: 1(0.0) - 1(0.0) - 1(0.0) - 1(0.0) - 1(0.0)
```

Here, vehicle 1 leaves garage 1, loads 1344 units of product 0 at depot 1, delivers them to station 2, and returns home. Vehicle 2 loads 8947 units of product 1, serves stations 1, 2, and 3, then returns to garage 1. Neither vehicle changes configuration, so their cumulative transition costs remain zero.

4. Summary metrics

After the last vehicle block, the file ends with six lines:

```
2
7
55.66
1385.07
Intel Core i7-10700K
0.245
```

They contain, in this order:

1. **Vehicles used** — the number of vehicles that perform at least one delivery.
2. **Product transitions** — the total number of charged product transitions.
3. **Total transition cost** — the sum of all preparation and loading-related transition costs.
4. **Total distance** — the Euclidean distance traveled by the complete fleet.
5. **Processor** — the processor used to produce the solution.
6. **Resolution time** — the computation time in seconds.

5. Feasibility requirements

A valid solution must respect all of the following conditions:

- each vehicle appears at most once;
- every route starts and ends at that vehicle's home garage;
- each trip begins with a positive depot load and includes at least one delivery;
- a vehicle carries one product throughout a trip, with a new product selected only when loading at a depot;
- the quantity loaded for a trip equals the quantity delivered and does not exceed vehicle capacity;
- depot stocks remain non-negative;

- every station-product demand is met exactly;
- one vehicle serves a given station-product pair at most once across all its trips; it may revisit the same station only to deliver another product;
- cumulative transition costs and final metrics match the routes described in the file.